Python Virtual Environmnet - Pro Stack Academy

By Narasimha Reddy

A Python virtual environment on Windows provides an **isolated space** for Python projects, allowing for **independent management of dependencies** and **packages** without affecting the global Python installation or other projects.

This is achieved through the built-in `venv` module.

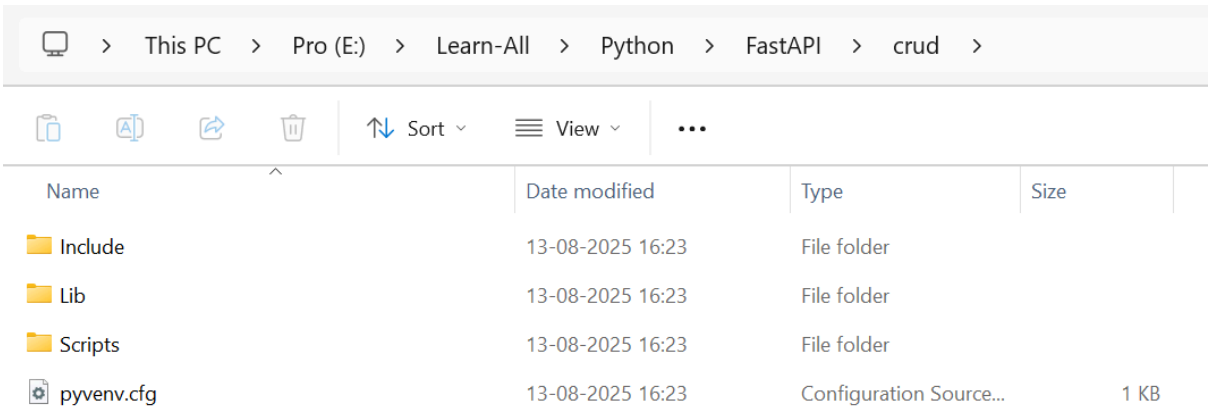
Creating a Virtual Environment:

- **Open a terminal:** Use Command Prompt or PowerShell.
- **Navigate to your project directory:** Use the `cd` command to move to the desired location where you want to create the virtual environment.
- **Create the virtual environment:** Execute the following command, replacing `myenv` with your desired environment name:

Code

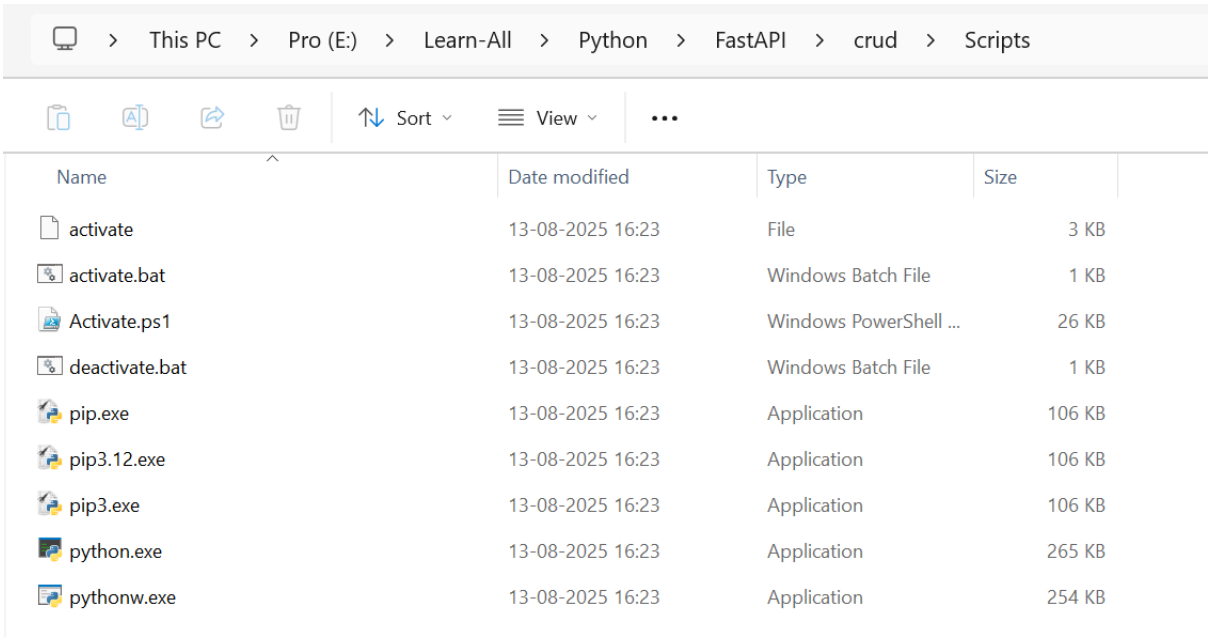
```
python -m venv myenv
```

```
E:\Learn-All\Python\FastAPI>python -m venv crud
```



Python Virtual Environmnet - Pro Stack Academy

By Narasimha Reddy



This creates a new directory named `myenv` (or whatever name you choose) containing a self-contained Python installation and a `Scripts` folder.

Activating the Virtual Environment:

- **Activate the environment:** From within your project directory, execute the activation script located in the `Scripts` folder of your virtual environment.
  - **For Command Prompt:**

```
myenv\Scripts\activate.bat
```

The primary advantage of using a virtual environment in Python projects is **dependency isolation**.

A virtual environment creates an **isolated Python environment** that contains its **own installation directories** and doesn't share libraries with other virtual environments or the system Python installation. This means:

- **No dependency conflicts**: Different projects can use different versions of the same package without interfering with each other
- **Clean project dependencies**: You can install exactly what each project needs without worrying about breaking other projects

- **Reproducible environments**: Other developers can recreate the exact same environment using your requirements file
- **System protection**: You won't accidentally modify or break your system's Python installation.

Imagine you're working on **two Python projects**:

- **Project A** needs **Django 3.2**
- **Project B** needs **Django 4.1**

If you install Django directly on your computer (system-wide), you can only have **one version at a time**.
So when you switch projects, one of them might **stop working** because it needs a different version.

A **virtual environment** fixes this problem.

It's like giving **each project its own mini-Python setup** — with its own packages and versions.

So:

- Project A's virtual environment can have Django 3.2
- Project B's virtual environment can have Django 4.1

And both work perfectly fine, without interfering with each other.

<u>You can create virtual environments using:</u>

1. `venv` (comes with Python)
2. `virtualenv` (older but popular)
3. `conda` (used in data science)

A virtual environment is a **separate space for each Python project** so you can use different libraries and versions safely.

Python Virtual Environmnet - Pro Stack Academy

By Narasimha Reddy